Bob's First Attempt

Bob's library maps each class to a database table and each object to a record. When Bob creates an object or accesses its attributes, the object generates a string of SQL and sends it to the database. All this functionality is wrapped in a class:

the_m_word/orm.rb

```
class Entity
  attr_reader :table, :ident

  def initialize(table, ident)
    @table = table
    @ident = ident
    Database.sql "INSERT INTO #{@table} (id) VALUES (#{@ident})"
  end

  def set(col, val)
    Database.sql "UPDATE #{@table} SET #{col}='#{val}' WHERE id=#{@ident}"
  end

  def get(col)
    Database.sql ("SELECT #{col} FROM #{@table} WHERE id=#{@ident}")[0][0]
  end
end
```

In Bob's database, each table has an id column. Each Entity stores the content of this column and the name of the table to which it refers. When Bob creates an Entity, the Entity saves itself to the database. Entity#set generates SQL that updates the value of a column, and Entity#get generates SQL that returns the value of a column. (In case you care, Bob's Database class returns recordsets as arrays of arrays.)

Bob can now subclass Entity to map to a specific table. For example, class Movie maps to a database table named movies:

```
class Movie < Entity
  def initialize(ident)
    super "movies", ident
  end

  def title
    get "title"
  end

  def title=(value)
    set "title", value
  end
```